DPDP COMPLIANCE AUDIT

saas-copilot

Digital Personal Data Protection Act 2023

64/100

Grade C — Needs Work

Scanned: 24 March 2026

Files indexed: 45

Languages: FastAPI, SQLAlchemy, Pydantic, python-jose, passlib, uvicorn

CONFIDENTIAL — Internal Use Only

EXECUTIVE SUMMARY

The application collects user name, email, and password hash. The biggest DPDP risk is the potential for unauthorized access to user credentials due to the storage of password hashes and the use of JWTs, without explicit consent mechanisms visible in the code.

Validation Summary Rule findings: 11 confirmed, 3 false positives suppressed • AI gaps: 1 dual-validated • Score: 65 (rules) → 64 (integrated)

Severity	Count	Status
HIGH	4	Requires Action
MEDIUM	2	Requires Action
INFO	3	Advisory/Info
PASS	1	Advisory/Info

Tech Stack: FastAPI, SQLAlchemy, Pydantic, python-jose, passlib
Risk Profile: The application collects user name, email, and password hash. The biggest DPDP risk is the potential for unauthorized access to user credentials due to the storage of password hashes and the use of JWTs, without explicit consent mechanisms visible in the code.

Compliance score by section

DPDP Section	Weight	Score	Status
Section 6	22	11/22 (50%)	WARN
Section 6(6)	8	0/8 (0%)	FAIL
Section 8(1)	18	18/18 (100%)	PASS
Section 8	10	10/10 (100%)	PASS
Section 8(3)	8	8/8 (100%)	PASS
Section 8(4)	8	0/8 (0%)	FAIL
Section 8(6)	8	6.8/8 (85%)	PASS
Section 9	6	6/6 (100%)	PASS
Section 11	6	2.1/6 (35%)	FAIL
Section 16	4	4/4 (100%)	PASS
Section 5	2	2/2 (100%)	PASS
Section 13 — Grievance Redressal	4	0/4 (0%)	FAIL

Estimated Total Remediation Effort: 2.6–9.1 engineering days

Data Flow Analysis

The following PII flow signals were inferred via static analysis (import/call adjacency). These are heuristic code-path chains, not proven runtime traces. Use them to guide developer review (confirm

actual request → handler → service → database paths in your app).

Flow 1: Application Logs

PII fields: email, password

```
Login.jsx → api.js
```

Flow 2: Application Logs

PII fields: email, password

```
Signup.jsx → api.js
```

Rule	Section	Severity	File
CONSENT_MISSING_REPO_LEVEL	Section 6 — Consent	HIGH	REPO-WIDE
CONSENT_WITHDRAWAL_MISSING	Section 6(6) — Consent Withdrawal	HIGH	N/A
AUDIT_TRAIL_MISSING	Section 8(4) — Audit Trail	HIGH	N/A
GRIEVANCE_OFFICER_MISSING	Section 13 — Grievance Redressal	HIGH	N/A
DATA_ACCESS_MISSING	Section 11 — Right to Access	MEDIUM	N/A
NO_BREACH_ALERTING	Section 8(6) — Breach	LOW	N/A
DATA_PORTABILITY_MISSING	Section 11 — Data Portability	LOW	N/A
NO_THIRD_PARTY_DETECTED	Section 5 — Notice	INFO	N/A
NO_THIRD_PARTY_DATA_SHARING	Section 5 — Purpose Limitation	INFO	N/A
NO_CROSS_BORDER_DETECTED	Section 16 — Cross-Border Transfer	INFO	N/A
CONSENT_PRESENT	Section 6 — Consent	PASS	backend/app/api/auth.py
	Section 8(5) — Data Protection	MEDIUM	

30-Day Remediation Roadmap

The following plan prioritises findings by severity and estimated effort. Complete Week 1 items before your next security review.

Week 1 — Critical (10–36 hours)

Fix immediately — these represent the highest legal exposure.

- ■ Consent Missing Repo Level — REPO-WIDE (4–16 hrs)
- ■ Consent Withdrawal Missing (1–4 hrs)
- ■ Audit Trail Missing (4–12 hrs)
- ■ Grievance Officer Missing (1–4 hrs)

Week 2 — High Priority (4–14 hours)

Address within 2 weeks — short fixes with significant compliance impact.

- ■ Data Access Missing (2–6 hrs)
- ■ (2–8 hrs)

Week 4 — Advisory (4–14 hours)

Address before next compliance review.

- ■ No Breach Alerting (2–6 hrs)
- ■ Data Portability Missing (2–8 hrs)

Total estimated remediation effort: 18–64 hours (2.2–8.0 engineering days)

Changes Since 2026-03-24

▼ Score: 65.0 → 64 (-1.0 points)

✓ 1 finding(s) resolved
■ 3 new finding(s) introduced
— 11 finding(s) unchanged

Resolved since last scan:

- ✓

New findings this scan:

- ■ Password Not Hashed [HIGH]
- ■ No Deletion Mechanism [MEDIUM]
- ■ Retention Missing [MEDIUM]

DETAILED FINDINGS

Finding 1 of 12

[Section 6 — Consent]

1. [HIGH] CONSENT_MISSING_REPO_LEVEL — CODEBASE-WIDE:44

Evidence snippet: The `/admin/ingest` endpoint is for administrative ingestion and requires an API key, not user consent.

Estimated fix time: 4–16 hours

No consent mechanism detected anywhere in the repository.

Risk:

The absence of a consent mechanism means the application cannot obtain or manage user consent for data processing, which is a direct violation of DPDP Section 6. This can lead to unauthorized data processing and significant legal penalties.

Fix:

1. Create a new SQLAlchemy model `Consent` with fields for user ID, consent type, timestamp, and status (granted/revoked).

2. Implement a new FastAPI endpoint (e.g., `/consent`) to allow users to grant or revoke consent for specific data processing activities.

3. Modify existing data processing flows to check for and respect user consent before processing personal data.

4. Add a new API endpoint (e.g., `/users/me/consent`) to allow users to view and manage their consent preferences.

DPDP:

Section 6 — Consent requires data subjects to give consent for the processing of their personal data.

Example:

```
from pydantic import BaseModel from datetime import datetime class ConsentBase(BaseModel): consent_type: str timestamp: datetime = datetime.utcnow() status: str # 'granted' or 'revoked' class ConsentCreate(ConsentBase): pass class Consent(ConsentBase): user_id: int class Config: orm_mode = True
```

Finding 2 of 12

[Section 6(6) — Consent Withdrawal]

2. [HIGH] CONSENT_WITHDRAWAL_MISSING — N / A

Estimated fix time: 1–4 hours

Consent is collected but no withdrawal mechanism detected. DPDP Section 6(6) requires consent withdrawal to be as easy as giving consent. Users must be able to revoke consent at any time.

Risk:

The application lacks a mechanism for users to withdraw their consent, which is a mandatory requirement under DPDP Section 6(6). Failure to provide an easy withdrawal process can result in continued unauthorized processing and legal repercussions.

Fix:

1. Create a new API endpoint in `backend/app/api/consent.py` (e.g., `DELETE /consent/{consent_id}`) to handle consent withdrawal requests.

2. Implement logic in the endpoint to mark the specified consent as 'revoked' in the database.

3. Update the frontend `ChatPage` or a new `SettingsPage` component to include a 'Manage Consent' section.

4. Add a button or link in the 'Manage Consent' section that calls the new withdrawal API endpoint when clicked by the user.

DPDP:

Section 6(6) — Data subjects have the right to withdraw their consent at any time, and this withdrawal must be as easy as giving consent.

Example:

```
from fastapi import APIRouter, Depends, HTTPException from sqlalchemy.orm import Session from app.db.session import SessionLocal from app.schemas.consent import ConsentUpdate router = APIRouter(prefix="/consent", tags=["Consent"]) def get_db(): db = SessionLocal() try: yield db finally: db.close() @router.delete("/{consent_id}") def withdraw_consent(consent_id: int, db: Session = Depends(get_db)): # Logic to find and revoke consent by consent_id
```

Finding 3 of 12

[Section 8(4) — Audit Trail]

3. [HIGH] AUDIT_TRAIL_MISSING — N / A

Estimated fix time: 4–12 hours

No audit trail infrastructure detected. DPDP Section 8(4) requires complete records of personal data processing activities. No access logging, audit model, or change tracking found in the codebase.

Risk:

The absence of a comprehensive audit trail means there are no records of personal data processing activities, violating DPDP Section 8(4). This lack of accountability makes it impossible to track data access, modifications, or deletions, increasing the risk of data misuse and non-compliance.

Fix:

1. Create a new SQLAlchemy model `AuditLog` with fields for timestamp, user ID, action type (e.g., 'CREATE', 'READ', 'UPDATE', 'DELETE'), target data, and details.

2. Implement a logging function in `backend/app/utils/audit_logger.py` that creates `AuditLog` entries.

3. Integrate the audit logging function into relevant API endpoints (e.g., user management, data ingestion) to record key processing activities.

4. Modify the `log_requests` middleware to capture more granular request details, including user ID if authenticated, and log them as 'READ' actions.

DPDP:

Section 8(4) — Data fiduciaries must maintain complete records of all personal data processing activities.

Example:

```
from datetime import datetime from sqlalchemy import Column, Integer, String, DateTime, ForeignKey from app.db.base_class import Base class AuditLog(Base): __tablename__ = "audit_logs" id = Column(Integer, primary_key=True, index=True) timestamp = Column(DateTime, default=datetime.utcnow) user_id = Column(Integer, ForeignKey("users.id"), nullable=True) action = Column(String, index=True) target_data = Column(String) details = Column(String, nullable=True)
```

Finding 4 of 12

[Section 13 — Grievance Redressal]

4. [HIGH] GRIEVANCE_OFFICER_MISSING — N / A

Estimated fix time: 1–4 hours

No compliant Grievance Officer / DPO publication found. A PASS requires grievance-related text or a dedicated route **and** a contact email or dedicated grievance/DPO URL. Tiny repos without a grievance route are not marked PASS.

Risk:

The absence of a designated grievance officer or a clear process for handling complaints means users have no recourse for addressing concerns about their data, violating DPDP Section 13. This can lead to unresolved issues, reputational damage, and potential regulatory action.

Fix:

1. Designate a specific individual or role within the organization as the Grievance Officer responsible for handling data subject complaints.

2. Add a new API endpoint (e.g., `POST /grievances`) to allow users to submit their grievances, including details of their complaint and contact information.

3. Create a `Grievance` SQLAlchemy model to store submitted grievances, including fields for user ID, complaint details, submission date, and status.

4. Update the application's contact or help section (frontend) to clearly display the contact details or a link to the grievance submission form.

DPDP:

Section 13 — Data fiduciaries must establish a mechanism for data subjects to address grievances regarding their personal data.

Example:

```
from fastapi import APIRouter, Depends, HTTPException from sqlalchemy.orm import Session from app.db.session import SessionLocal from app.schemas.grievance import GrievanceCreate router = APIRouter(prefix="/grievances", tags=["Grievances"]) def get_db(): db = SessionLocal() try: yield db finally: db.close() @router.post("/") def submit_grievance(payload: GrievanceCreate, db: Session = Depends(get_db)): # Logic to create and store a new grievance record
```

Finding 5 of 12

[Section 11 — Right to Access]

5. [MEDIUM] DATA_ACCESS_MISSING — N / A

Estimated fix time: 2–6 hours

No endpoint detected that allows authenticated users to retrieve their own personal data. Under DPDP Section 11, Data Principals have the right to access information about their personal data being processed.

Risk:

The absence of a data access endpoint means users cannot exercise their right to access their personal data, violating DPDP Section 11 and potentially leading to non-compliance penalties.

Fix:

1. Create a new FastAPI endpoint (e.g., GET /users/me) that requires authentication.

2. Inside the endpoint, retrieve the authenticated user's ID from the token.

3. Query the database for the user's record using their ID.

4. Return the user's personal data in a JSON format, excluding sensitive fields like password hashes.

DPDP:

Section 11 — Right to Access: Data Principals have the right to access information about their personal data being processed.

Example:

```
from fastapi import APIRouter, Depends, HTTPException from app.core.security import
get_current_user from app.models.user import User router = APIRouter()
@router.get('/users/me') def read_users_me(current_user: User = Depends(get_current_user)): #
Assuming get_current_user returns the User object return { "id": current_user.id, "name":
current_user.name, "email": current_user.email }
```

Finding 6 of 12

[Section 8(6) — Breach]

6. [LOW] NO_BREACH_ALERTING — N / A

Estimated fix time: 2–6 hours

Structured logging detected but no breach notification mechanism found. DPDP Section 8(6) requires notifying affected individuals and the Authority without undue delay when a breach occurs. Logging alone is insufficient — implement automated alerting (Sentry, PagerDuty, or equivalent) with escalation to a compliance team.

Risk:

The application lacks automated breach notification mechanisms, meaning individuals and the Authority may not be promptly informed of data breaches as required by DPDP Section 8(6), delaying response and potentially increasing harm.

Fix:

1. Integrate a third-party error tracking and alerting service (e.g., Sentry, PagerDuty).

2. Configure the service to monitor exceptions and critical events within the application.

3. Set up alerts to notify a designated compliance team or individual upon detection of potential data breaches or significant security incidents.

4. Ensure logs from `backend/app/utils/logger.py` are ingested by the alerting system for context.

DPDP:

Section 8(6) — Breach: Data Fiduciaries must notify the Authority and affected Data Principals without undue delay when a personal data breach occurs.

Example:

```
import sentry_sdk sentry_sdk.init( dsn="YOUR_SENTRY_DSN", # Set traces_sample_rate to 1.0 to capture 100% of transactions for performance monitoring. # We recommend adjusting this value in production. traces_sample_rate=1.0, # If you wish to associate users to events, uncomment the following line # and add a user identifier into the event's context. # send_default_pii=True ) # In your FastAPI app: # from fastapi import FastAPI # app = FastAPI() # @app.get("/error") # async def trigger_error(): # raise ValueError("This is a test error")
```

Finding 7 of 12

[Section 11 — Data Portability]

7. [LOW] DATA_PORTABILITY_MISSING — N / A

Estimated fix time: 2–8 hours

No data export or portability endpoint detected. DPDP Section 11 requires Data Fiduciaries to provide personal data in a commonly used, machine-readable format upon request.

Risk:

The absence of a data portability endpoint prevents users from exercising their right to receive their personal data in a usable format, failing to comply with DPDP Section 11 and hindering data portability.

Fix:

1. Create a new FastAPI endpoint (e.g., GET /users/me/export) that requires authentication.

2. Inside the endpoint, retrieve the authenticated user's ID.

3. Fetch all relevant personal data for the user from the database.

4. Format the data into a common, machine-readable format (e.g., JSON, CSV).

5. Return the data with appropriate headers for file download (e.g., Content-Disposition).

DPDP:

Section 11 — Data Portability: Data Fiduciaries must provide personal data in a commonly used, machine-readable format upon request.

Example:

```
from fastapi import APIRouter, Depends, HTTPException, Response from app.core.security import get_current_user from app.models.user import User import json router = APIRouter() @router.get('/users/me/export') def export_users_me(current_user: User = Depends(get_current_user)): user_data = { "id": current_user.id, "name": current_user.name, "email": current_user.email # Add other relevant personal data fields here } content = json.dumps(user_data, indent=2) return Response(content=content, media_type='application/json', headers={"Content-Disposition": "attachment; filename=user_data.json"})
```

Finding 8 of 12

[Section 5 — Notice]

8. [INFO] NO_THIRD_PARTY_DETECTED — N / A

No known third-party analytics or tracking SDKs detected in codebase.

Finding 9 of 12

[Section 5 — Purpose Limitation]

9. [INFO] NO_THIRD_PARTY_DATA_SHARING — N / A

No third-party data sharing detected.

Finding 10 of 12

[Section 16 — Cross-Border Transfer]

10. [INFO] NO_CROSS_BORDER_DETECTED — N / A

No foreign cloud or non-India region config detected.

Finding 11 of 12

[Section 6 — Consent]

11. [PASS] CONSENT_PRESENT — backend / app / api / auth.py

Auth enforcement pattern detected inline — consent likely handled at registration.

Finding 12 of 12

[Section 8(5) — Data Protection]

12. [MEDIUM] — CODEBASE-WIDE

Missing encryption at rest for sensitive PII

AI-Identified Compliance Gaps

The following gaps were identified by AI analysis and cross-validated. These are advisory observations, not verified rule violations. Human review is required before acting on these findings.

[AI-INFERRED] Missing encryption at rest for sensitive PII Confidence: 57%

Section 8(5) — Data Protection

The `backend/app/models/user.py` defines the `User` model with fields for `name` and `email`. While `password_hash` is hashed, there is no indication in the repository context that these `name` and `email` fields, which contain personal data, are encrypted at rest in the database.

Recommendation: Implement encryption at rest for sensitive PII fields such as `name` and `email` in the `User` model within `backend/app/models/user.py`. This can be achieved through database-level encryption or application-level encryption prior to storage.